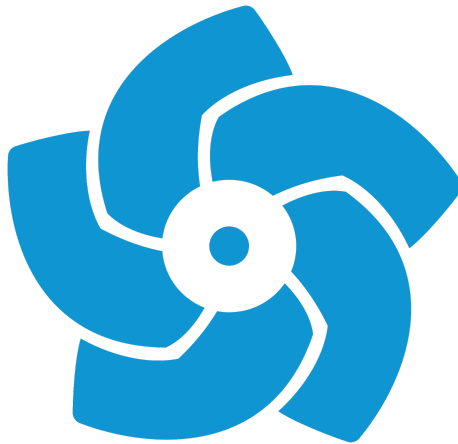


BILLMATE - Smart Split Bill App

WACANA



Identitas Kelompok:

25.12.3691 - Yulianda Laia

25.12.3442 - Muhammad Rifky Putra Asisyaeft

AMIKOM COMPUTER CLUB

WEB PROGRAMMING

DIVISION

2026

DAFTAR ISI

BAB I.....	3
PENDAHULUAN.....	3
I.1. Latar Belakang.....	3
I.2. Pernyataan Masalah.....	3
I.3. Tujuan Proyek.....	4
BAB II.....	5
ISI.....	5
II.1. Tech Stack.....	5
II.2. UI/UX Design.....	5
II.2.1. Wireframe.....	5
II.2.2. Mockup / High-Fidelity Design.....	5
II.3. Database Design & API Specification.....	5
II.3.1. Entity Relationship Diagram (ERD).....	5
II.3.2. API Specification.....	5
II.4. Frontend Development.....	6
II.4.1. Struktur Project.....	6
II.4.2. Implementasi Halaman & Komponen.....	6
II.4.3. Integrasi API.....	6
II.5. Backend Development.....	6
II.5.1. Struktur Project.....	6
II.5.2. Implementasi API Endpoint.....	6
II.5.3. Authentication & Authorization.....	6
II.5.4. Middleware & Security.....	7
II.6. Tautan Repository.....	7

BAB I

PENDAHULUAN

I.1. Latar Belakang

Kegiatan berkumpul dan makan bersama, seperti momen buka puasa bersama (*iftar*) maupun agenda kasual kelompok lainnya, telah menjadi bagian dari dinamika sosial yang melekat kuat di kalangan mahasiswa dan komunitas. Dalam praktiknya, guna menyederhanakan proses transaksi di kasir restoran, kelompok biasanya menerapkan sistem talangan di mana satu orang yang bertindak sebagai *host* akan membayar seluruh total nota terlebih dahulu. Namun, situasi penagihan kembali setelah acara selesai saat ini masih dihadapkan pada berbagai kendala teknis dan keterbatasan pemenuhan kebutuhan yang belum optimal. Keterbatasan retensi memori anggota kelompok membuat mereka sering kali lupa atau tidak mencatat secara rinci menu spesifik serta kuantitas makanan yang telah mereka konsumsi. Kondisi ini diperumit oleh adanya komponen tambahan pada nota restoran seperti Pajak Restoran (PPN), biaya pengemasan (*packing*), hingga biaya layanan (*service charge*) yang bervariasi, sementara metode perhitungan yang digunakan saat ini masih mengandalkan alat hitung konvensional (kalkulator ponsel) yang tidak mendukung otomatisasi pembagian biaya secara proporsional.

Ketiadaan sistem kalkulasi yang memadai ini menimbulkan berbagai dampak negatif yang secara signifikan menurunkan efisiensi waktu, di mana *host* harus menghabiskan waktu cukup lama hanya untuk memilah dan menghitung rincian utang setiap anggota kelompok satu per satu secara manual. Selain penurunan efisiensi, proses perhitungan manual yang rumit, terutama saat mencoba membagi potongan diskon atau pajak restoran secara adil berdasarkan harga makanan tiap individu—juga membawa risiko kesalahan hitung (*human error*) yang sangat tinggi sehingga rentan menghasilkan selisih angka yang merugikan salah satu pihak. Dalam jangka panjang, ketidakakuratan serta kurangnya transparansi dalam pembagian nominal tagihan ini sering kali menimbulkan rasa sungkan, kecurigaan, hingga potensi konflik sosial atau drama kelompok di dalam grup komunikasi pasca-acara. Melihat besarnya dampak tersebut, terdapat tingkat urgensi yang tinggi untuk menghadirkan sebuah sistem alat bantu hitung digital yang mampu mengotomatisasi kalkulasi ini secara cepat, akurat, dan transparan demi memotong birokrasi penagihan manual yang melelahkan.

Dalam upaya menyelesaikan permasalahan tersebut, proyek ini awalnya sempat mempertimbangkan pendekatan aplikasi berbasis *mobile* (Android/iOS). Namun, pendekatan tersebut dinilai kurang efektif karena karakteristik aplikasi *mobile* mewajibkan pengguna melewati proses pengunduhan serta instalasi yang berpotensi membebani kapasitas memori penyimpanan (*storage*) ponsel untuk sebuah platform yang intensitas penggunaannya hanya bersifat situasional atau berkala. Sebagai bentuk penyelesaian yang optimal, dipilihlah pendekatan solusi berbasis *Web Application* (Aplikasi Berbasis Web) bernama BillMate. Pemilihan arsitektur ini didasari oleh berbagai keunggulan strategis, seperti kemudahan akses lintas perangkat (*cross-platform*) di mana pengguna cukup membuka tautan melalui *browser* tanpa perlu mengunduh aplikasi, serta pengelolaan data yang terpusat sehingga sistem tetap ringan dan responsif. Selain itu, aplikasi berbasis web memberikan kemudahan pemeliharaan (*maintenance*) secara terpusat oleh pengembang dan sangat efektif dalam mendukung kebutuhan pengguna, terutama untuk mempermudah integrasi *output* data rincian nota yang dinamis agar dapat disalin atau dibagikan secara langsung ke platform pihak ketiga seperti WhatsApp dengan satu klik. Melalui pengembangan solusi berbasis web ini, BillMate diharapkan dapat memberikan manfaat fungsional yang konkret dalam mentransformasi aktivitas penagihan kelompok menjadi lebih efektif, efisien, dan adil.

I.2. Pernyataan Masalah

Adapun permasalahan yang menjadi fokus utama pada proyek ini dapat dijabarkan melalui beberapa poin berikut.

1. Proses pembagian tagihan makan bersama saat ini masih dilakukan secara manual dengan melihat nota fisik lalu menagihnya satu per satu lewat pesan teks
2. Kondisi manual tersebut membuat host kesulitan membagi biaya tambahan seperti pajak, diskon, dan ongkir secara adil berdasarkan pesanan tiap orang sehingga rentan salah hitung
3. Permasalahan utama yang menjadi fokus proyek ini adalah pembuatan aplikasi web yang bisa menghitung tagihan makan secara otomatis, adil, dan hasilnya bisa langsung dikirim ke WhatsApp

4. Pengembangan solusi ini dibatasi hanya pada input menu manual serta kalkulasi pembagian akhir melalui website tanpa perlu mengunduh aplikasi atau menggunakan sistem pemindaian nota otomatis

I.3. Tujuan Proyek

Beberapa tujuan proyek yang ingin dicapai melalui pengembangan sistem ini sebagai berikut.

1. Menghasilkan solusi permasalahan dengan membangun aplikasi web kalkulator pintar untuk menghitung pembagian tagihan makan secara otomatis dan adil
2. Meningkatkan efektivitas dan efisiensi dengan mempercepat waktu perhitungan biaya tambahan seperti pajak, diskon, dan ongkir secara proporsional dalam hitungan detik
3. Memenuhi kebutuhan pengguna melalui penyediaan aplikasi berbasis web yang ringan dan bisa langsung diakses lewat *browser* tanpa harus mengunduh aplikasi
4. Memberikan manfaat dan nilai tambah berupa hasil hitungan yang transparan bagi kelompok pertemanan untuk mencegah salah hitung dan menghindari drama kelompok
5. Menjadi dasar evaluasi keberhasilan untuk menguji apakah aplikasi ini benar-benar berjalan dengan baik dan sukses membantu pengguna saat dicoba langsung di lapangan

BAB II

ISI

II.1. Tech Stack

Pengembangan BillMate menggunakan arsitektur *multi-service* yang memisahkan tanggung jawab antara lapisan antarmuka pengguna (*frontend*), logika bisnis kalkulasi *backend*, serta sistem penyimpanan riwayat transaksi lokal browser. Pemisahan ini dipilih untuk memungkinkan pembagian kerja paralel yang efisien di dalam tim dan memastikan setiap komponen dapat dikembangkan serta diuji secara independen tanpa saling ketergantungan. Seluruh *service* di-deploy secara terintegrasi, di mana layanan *backend* berbasis Laravel di-deploy pada platform web hosting server sedangkan lapisan *frontend* berbasis HTML, Tailwind CSS, dan JavaScript vanilla berjalan secara statis. Tabel berikut merangkum teknologi yang digunakan dalam ekosistem BillMate beserta justifikasi pemilihannya.

Lapisan	Teknologi	Alasan Pemilihan
Frontend	HTML5 (Semantic HTML)	Sebagai fondasi struktural markup standar web untuk membangun elemen antarmuka halaman yang bersih, terstruktur secara semantik, serta menjamin aksesibilitas elemen input formulir.
	Tailwind CSS	<i>Utility-first CSS framework</i> untuk membangun komponen antarmuka nota digital dan dashboard room yang responsif. Memungkinkan kustomisasi tema warna serta konsistensi visual tinggi secara kilat tanpa memerlukan berkas stylesheet eksternal yang rumit.
	Vanilla JavaScript (ES6+)	Bahasa pemrograman murni untuk mengontrol seluruh logika aplikasi secara asinkron tanpa <i>overhead</i> dari <i>framework</i> berat. Digunakan untuk manipulasi DOM dinamis, kalkulasi biaya proporsional lokal, serta menjembatani komunikasi data dengan backend menggunakan Fetch API.
	Web Storage API (localStorage & sessionStorage)	Mekanisme penyimpanan lokal pada browser untuk manajemen status aplikasi (<i>state management</i>). <i>Session storage</i> digunakan untuk menyimpan data sesi pembuatan ruangan

		sementara, sedangkan <i>local storage</i> digunakan untuk menyimpan data hasil finalisasi tagihan agar dapat diakses kembali pada sistem riwayat secara luring.
	Animate On Scroll (AOS)	Pustaka animasi ringan berbasis CSS dan JavaScript untuk memberikan efek transisi visual dinamis (<i>fade</i> dan <i>zoom</i>) pada elemen halaman landing dan pembuatan room guna meningkatkan kualitas pengalaman pengguna (<i>user experience</i>).
	Google Material Symbols	Pustaka ikon berbasis font vektor terpadu untuk menyediakan elemen grafis pendukung antarmuka (seperti ikon kalkulasi, riwayat, dan salin) secara konsisten dengan beban pemuatan aset yang seminimal mungkin.
Backend	PHP 8.3	Bahasa pemrograman sisi server (<i>server-side</i>) yang modern, berkinerja tinggi, dan stabil. PHP 8.3 dipilih karena membawa peningkatan performa eksekusi dan efisiensi memori yang sangat baik untuk menangani request API secara cepat.
	Laravel 13.8	Framework PHP modern dengan arsitektur <i>slim application</i> yang efisien. Memudahkan pembuatan RESTful API melalui fitur routing yang ringkas, ORM Eloquent untuk pemetaan objek database, serta sistem validasi input terintegrasi guna memastikan keamanan data dari sisi klien.
	MySQL	Sistem Manajemen Database Relasional (RDBMS) yang handal, cepat, dan teruji. MySQL digunakan untuk mengelola basis data db_billmate dengan arsitektur ternormalisasi 3 tabel (<i>transactions</i> , <i>transaction_members</i> , dan <i>transaction_items</i>) untuk menjamin konsistensi data keuangan.
	Laravel Sanctum v4.0	Pustaka autentikasi bawaan Laravel yang ringan dan sangat aman untuk mengelola <i>Personal Access Token</i> . Diterapkan dalam proyek untuk mempersiapkan sistem keamanan token API di

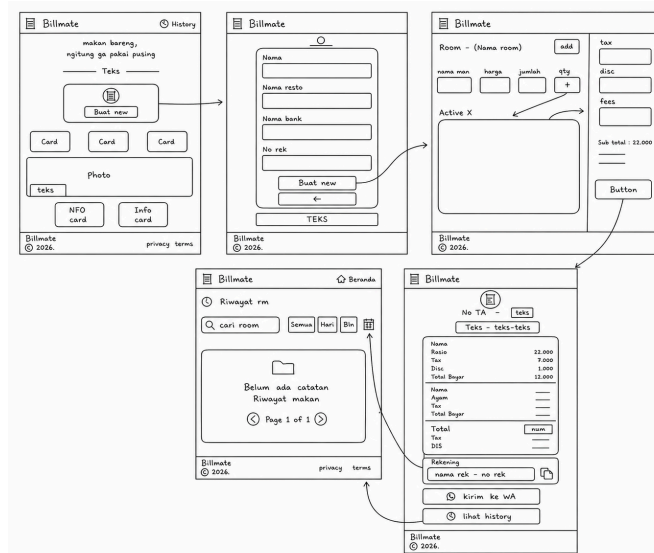
		masa mendatang tanpa membebani performa aplikasi.
--	--	---

II.2. UI/UX Design

II.2.1. Wireframe

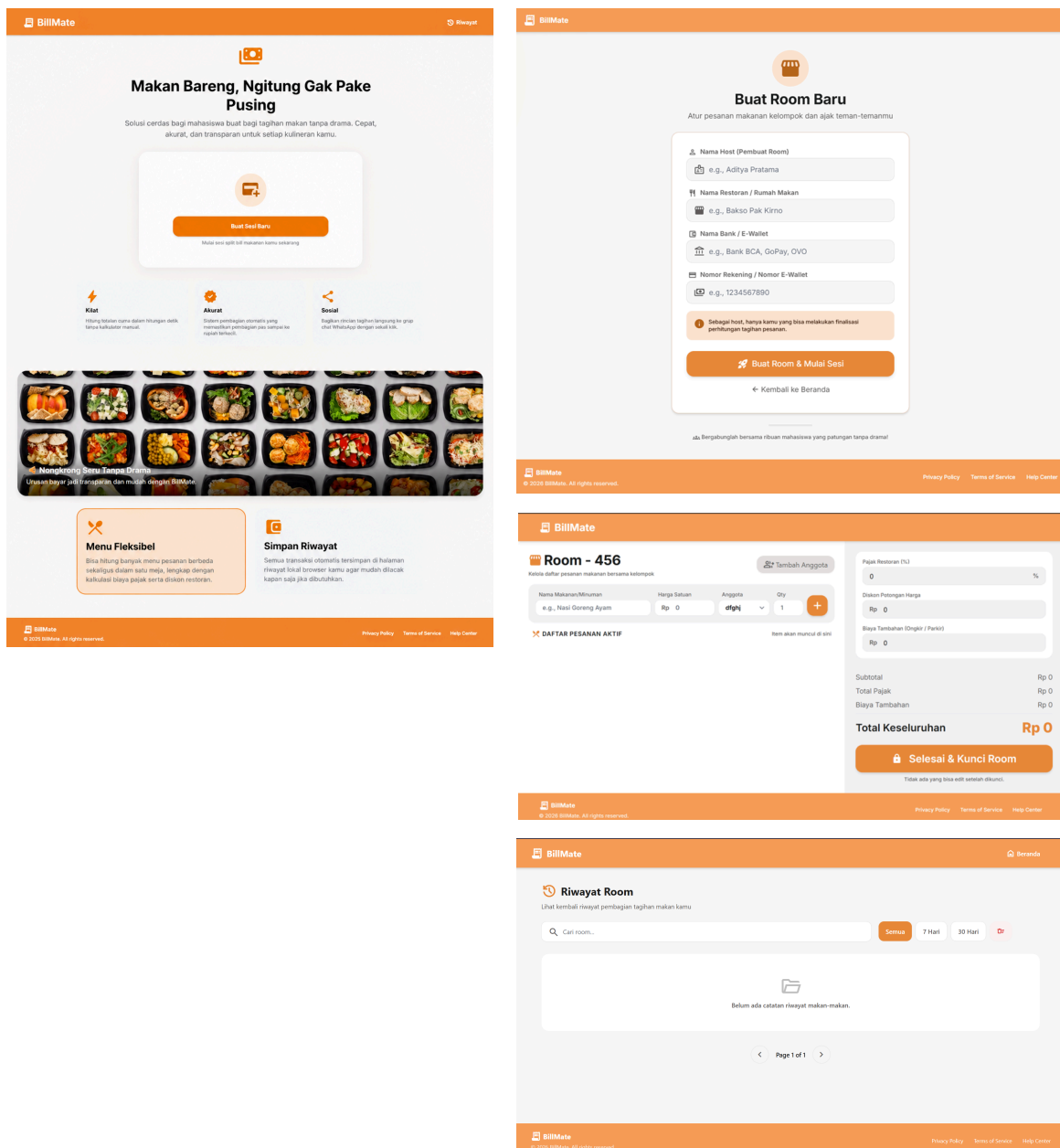
Siklus interaksi dan perpindahan halaman pada *frontend* BillMate dikendalikan secara dinamis berbasis aktivitas pengguna (*event-driven design*). Proses awal dimulai dari halaman Beranda (*index.html*) di mana pengguna diarahkan ke formulir Buat Room (*createroom.html*) saat mengeklik tombol aksi inisialisasi sesi baru. Setelah host melengkapi data identitas perbankan dan mengeklik "Buat Room", sistem memvalidasi kelengkapan input, mengunci variabel data ke dalam memori *sessionStorage*, lalu melempar pengguna ke Dashboard Room (*room.html*) untuk memulai perekaman menu makanan anggota meja.

Ketika proses input pesanan kelompok dirasa sudah lengkap, host dapat mengeklik tombol "Selesai & Kunci Room" di dalam dashboard aktif. Aksi tersebut memicu pengumpulan data pesanan menjadi satu objek payload JSON untuk dikirim secara asinkron ke server backend Laravel melalui gerbang *window.billMateAPI*. Setelah server berhasil menghitung pembagian biaya proporsional, respon sukses dikembalikan dan dicadangkan ke dalam *localStorage* global sebelum browser dialihkan secara permanen menuju halaman Rincian Nota (*nota.html*). Dari lembar nota digital ini, pengguna dapat mengakses halaman Riwayat Transaksi (*history.html*) melalui tombol pintasan di bagian bawah, yang mana halaman riwayat tersebut juga mendukung navigasi balik ke halaman nota secara instan engan memuat ulang data lama transaksi saat salah satu kartu arsip aktif diklik oleh pengguna.



II.2.2. Mockup / High-Fidelity Design

Rancangan antarmuka tingkat tinggi (*high-fidelity mockup*) merepresentasikan visualisasi final dari seluruh halaman utama aplikasi BillMate yang telah diimplementasikan secara presisi. Pembuatan komponen grafis dan penataan tata letak antarmuka ini dirancang secara digital menggunakan perangkat lunak **Figma** selaku *UI/UX design tool* utama, yang kemudian ditranslasikan secara langsung ke dalam bentuk kode menggunakan *utility-first CSS framework* dari **Tailwind CSS**. Setiap halaman dirancang secara responsif untuk memastikan kenyamanan visual serta konsistensi komponen antarmuka pengguna saat diakses melalui berbagai resolusi perangkat peramban (*web browser*).



II.3. Database Design & API Specification

II.3.1. Entity Relationship Diagram (ERD)

a. Gambaran Umum

Sistem Bill Mate menggunakan database relasional **MySQL** dengan nama database `db_billmate`. Skema database dirancang menggunakan pendekatan **normalisasi 3 tabel (3-Tier Normalized)** untuk menyimpan data transaksi pembagian tagihan (*split-bill*). Ketiga tabel utama yang menjadi fokus pembahasan adalah:

- i. **transactions** — Tabel induk (parent table)
- ii. **transaction_members** — Tabel anak pertama (child table 1)
- iii. **transaction_items** — Tabel anak kedua (child table 2)

Ketiga tabel ini saling terhubung melalui kolom **transaction_id** yang berfungsi sebagai *foreign key* pada tabel anak, mereferensikan kolom `transaction_id` pada tabel induk `transactions`.

b. Penjelasan Detail Setiap Tabel

1. Tabel transactions (Tabel Induk)

Tabel `transactions` berfungsi sebagai **tabel utama (parent table)** yang menyimpan data ringkasan keseluruhan dari setiap sesi pembagian tagihan. Setiap baris pada tabel ini merepresentasikan **satu sesi split-bill** yang dilakukan di sebuah restoran.

Kolom	Tipe Data	Constraint	Keterangan
<code>id</code>	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Identifikasi unik internal
<code>transaction_id</code>	VARCHAR(255)	UNIQUE	Kode transaksi yang di-generate secara unik dengan format QR-YYYYMMDD-XXXX
<code>restaurant_name</code>	VARCHAR(255)	NOT NULL	Nama restoran tempat makan

table_number	VARCHAR(255)	NOT NULL	Nomor meja di restoran
host_name	VARCHAR(255)	NOT NULL	Nama penanggungjawab/host yang akan menerima pembayaran
total_base_cost	DECIMAL(12,2)	NOT NULL	Total biaya pokok seluruh pesanan sebelum pajak, diskon, dan biaya tambahan
total_tax	DECIMAL(12,2)	NOT NULL	Total pajak yang dikenakan
total_discount	DECIMAL(12,2)	NOT NULL	Total diskon yang diberikan
total_extra_fees	DECIMAL(12,2)	NOT NULL	Total biaya tambahan (misal: biaya layanan)
grand_total	DECIMAL(12,2)	NOT NULL	Grand total akhir setelah perhitungan pajak, diskon, dan biaya tambahan
created_at	TIMESTAMP	NULLABLE	Waktu pembuatan record
updated_at	TIMESTAMP	NULLABLE	Waktu pembaruan record

2. Tabel transaction_members (Tabel Anak — Anggota)

Tabel transaction_members menyimpan **rincian pembagian biaya per individu/anggota** dalam suatu sesi transaksi. Setiap baris pada tabel ini merepresentasikan **satu anggota** beserta rincian tanggungan biayanya.

Relasi: Tabel ini memiliki hubungan **Many-to-One** dengan tabel transactions. Satu transaksi dapat memiliki **banyak anggota** (*one-to-many*). Foreign key transaction_id mereferensikan kolom transaction_id pada tabel transactions dengan aturan **ON DELETE CASCADE**, yang berarti jika data transaksi induk dihapus, maka seluruh data anggota terkait akan ikut terhapus secara otomatis.

Kolom	Tipe Data	Constraint	Keterangan
id	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Identifikasi unik internal

transaction_id	VARCHAR(255)	FOREIGN KEY → transactions.transaction_id	Referensi ke tabel transaksi induk
member_name	VARCHAR(255)	NOT NULL	Nama anggota kelompok
base_cost	DECIMAL(12,2)	NOT NULL	Total biaya pokok yang menjadi tanggungan anggota ini
tax_share	DECIMAL(12,2)	NOT NULL	Bagian pajak proporsional berdasarkan biaya pokok anggota
discount_share	DECIMAL(12,2)	NOT NULL	Bagian diskon proporsional berdasarkan biaya pokok anggota
extra_fee_share	DECIMAL(12,2)	NOT NULL	Bagian biaya tambahan (dibagi rata ke seluruh anggota)
grand_total	DECIMAL(12,2)	NOT NULL	Grand total yang harus dibayar oleh anggota ini
created_at	TIMESTAMP	NULLABLE	Waktu pembuatan record
updated_at	TIMESTAMP	NULLABLE	Waktu pembaruan record

3. Tabel transaction_items (Tabel Anak — Item Pesanan)

Tabel transaction_items menyimpan **detail setiap item/menu yang dipesan** dalam suatu sesi transaksi. Setiap baris merepresentasikan **satu item pesanan** beserta informasi harga, kuantitas, dan siapa anggota yang memesannya.

Relasi: Sama seperti tabel transaction_members, tabel ini memiliki hubungan **Many-to-One** dengan tabel transactions. Satu transaksi dapat memiliki **banyak item pesanan** (*one-to-many*). Foreign key transaction_id juga menerapkan aturan **ON DELETE CASCADE**.

Kolom	Tipe Data	Constraint	Keterangan
id	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Identifikasi unik internal
transaction_id	VARCHAR(255)	FOREIGN KEY → transactions.transaction_id	Referensi ke tabel transaksi induk

item_name	VARCHAR(255)	NOT NULL	Nama menu/item yang dipesan
price	DECIMAL(12,2)	NOT NULL	Harga satuan item
qty	INTEGER	NOT NULL	Jumlah/kuantitas item yang dipesan
total_price	DECIMAL(12,2)	NOT NULL	Total harga (price × qty)
user_name	VARCHAR(255)	NOT NULL	Nama anggota yang memesan item ini
created_at	TIMESTAMP	NULLABLE	Waktu pembuatan record
updated_at	TIMESTAMP	NULLABLE	Waktu pembaruan record

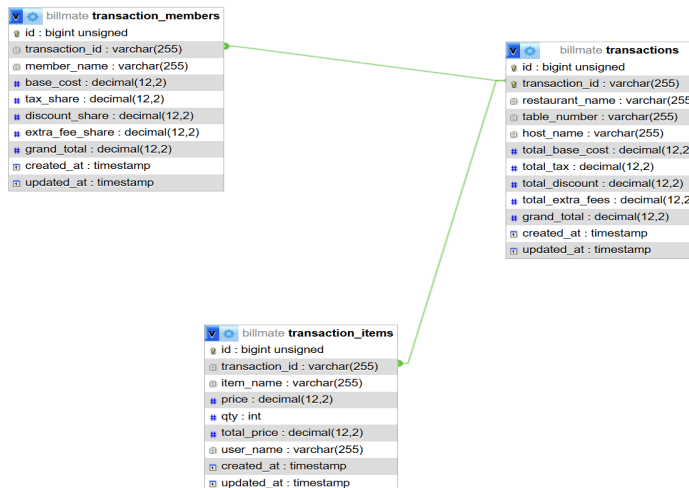
Hubungan Antar Tabel (Relationship Summary)

Ketiga tabel di atas membentuk struktur relasi **One-to-Many** sebagai berikut:

- **transactions** → **transaction_members**: Satu transaksi memiliki banyak anggota (1:N). Setiap anggota menyimpan rincian berapa besar tanggungan biaya yang harus dibayarnya (biaya pokok, bagian pajak, bagian diskon, bagian biaya tambahan, dan grand total individual).
- **transactions** → **transaction_items**: Satu transaksi memiliki banyak item pesanan (1:N). Setiap item menyimpan detail menu yang dipesan termasuk harga satuan, kuantitas, total harga, serta nama anggota yang memesan item tersebut.

Kedua tabel anak dihubungkan ke tabel induk melalui kolom `transaction_id` (bertipe VARCHAR/STRING) yang merupakan kode unik transaksi dengan format QR-YYYYMMDD-XXXX. Mekanisme **ON DELETE CASCADE** diterapkan pada kedua *foreign key*, sehingga integritas data tetap terjaga — ketika sebuah record transaksi dihapus dari tabel induk, seluruh record anggota dan item yang terkait juga akan dihapus secara otomatis.

Seluruh kolom desimal menggunakan tipe DECIMAL (12,2) untuk menjamin akurasi perhitungan keuangan hingga 12 digit dengan 2 angka di belakang koma.



II.3.2. API Specification

1. Base URL

http://localhost:8000/api

2. Daftar Endpoint

Aplikasi Bill Mate menyediakan **2 (dua) endpoint API** utama yang semuanya menggunakan metode HTTP **POST**:

1. Kalkulasi Split Bill

Properti	Detail
Endpoint	POST /api/split-bill/calculate
Deskripsi	Memproses kalkulasi pembagian tagihan secara adil, menyimpan data ke database secara relasional, dan mengembalikan hasil perhitungan
Content-Type	application/json
Autentikasi	Tidak diperlukan (endpoint publik)

Request Body:

```
{
  "restaurantName": "string (wajib, maks 255 karakter)",
  "tableNumber": "string (wajib, maks 50 karakter)",
  "hostName": "string (opsional, maks 100 karakter)",
  "bankName": "string (opsional, maks 100 karakter)",
  "rekening": "string (opsional, maks 100 karakter)",
  "members": ["string (wajib, min 1 anggota)"],
  "items": [
    {
      "name": "string (nama menu)",
      "price": "number (harga satuan)",
      "qty": "number (kuantitas)",
      "user": "string (nama pemesan)"
    }
  ],
  "additionalCosts": {
    "taxPercent": "number (opsional, 0-100)",
    "discount": "number (opsional, min 0)",
    "extraFees": "number (opsional, min 0)"
  }
}
```

Response Sukses (200):

```
{
  "success": true,
  "message": "Proses pembagian tagihan adil telah selesai dihitung dan disimpan secara relasional!",
  "data": {
    "restaurantName": "Restoran ABC",
    "tableNumber": "12",
    "transactionId": "QR-20260705-1234",
    "date": "Jul 05, 2026",
    "summary": {
      "totalBaseCost": 150000,
      "totalTax": 16500,
      "totalDiscount": 10000,
      "totalExtraFees": 5000,
      "grandTotal": 161500
    },
    "membersBreakdown": [
      {
        "memberName": "Andi",
        "baseCost": 75000,
        "taxShare": 8250,
        "discountShare": 5000,
        "extraFeeShare": 2500,
        "grandTotal": 80750,
        "orderedItems": [
          { "name": "Nasi Goreng", "price": 25000, "qty": 3, "total": 75000 }
        ]
      }
    ],
    "transferInfo": {
      "hostName": "Andi",
      "paymentOptions": [
        { "provider": "BCA", "accountNumber": "1234567890" }
      ]
    }
  }
}
```

Response Error Validasi (422):

```
{
  "success": false,
  "message": "Input data tidak valid sesuai format QRoom.",
  "errors": { "restaurantName": ["The restaurant name field is required."] }
}
```

2. Arsipkan Room/Sesi

Properti	Detail
Endpoint	POST /api/split-bill/archive

Deskripsi	Mengarsipkan data sesi transaksi untuk keperluan riwayat di sisi klien
Content-Type	application/json
Autentikasi	Tidak diperlukan (endpoint publik)

Request Body:

```
{
  "restaurantName": "string (wajib, maks 255 karakter)",
  "grandTotal": "number (wajib, min 0)",
  "members": ["string (wajib, min 1 anggota)"]
}
```

Response Sukses (200):

```
{
  "success": true,
  "message": "Room berhasil dikunci. Sesi ini telah resmi diarsipkan.",
  "archive": {
    "transactionId": "#QR-2026-A1B2",
    "restaurantName": "Restoran ABC",
    "date": "Jul 05, 2026",
    "timestamp": 1751721600,
    "totalAmount": 161500,
    "membersCount": 3,
    "membersPreview": ["Andi", "Budi", "Citra"],
    "status": "COMPLETED"
  }
}
```

Response Error Validasi (422):

```
{
  "success": false,
  "message": "Gagal mengarsipkan sesi. Struktur data tidak valid.",
  "errors": { "restaurantName": ["The restaurant name field is required."] }
}
```

Ringkasan Aturan Validasi

Field	Aturan	Endpoint
restaurantName	Wajib, string, maks 255 karakter	Keduanya
tableNumber	Wajib, string, maks 50 karakter	Calculate
hostName	Opsional, string, maks 100 karakter	Calculate

bankName	Opsional, string, maks 100 karakter	Calculate
rekening	Opsional, string, maks 100 karakter	Calculate
members	Wajib, array, min 1 elemen	Keduanya
members.*	Wajib, string, maks 100 karakter	Keduanya
items	Wajib, array, min 1 elemen	Calculate
additionalCosts.taxPercent	Opsional, numerik, 0–100	Calculate
additionalCosts.discount	Opsional, numerik, min 0	Calculate
additionalCosts.extraFees	Opsional, numerik, min 0	Calculate
grandTotal	Wajib, numerik, min 0	Archive

II.4. Frontend Development

II.4.1. Struktur Project

Pengembangan komponen antarmuka dan pengelolaan berkas pada sisi *frontend* BillMate diorganisasikan menggunakan pola struktur berbasis **Feature-Based Architecture** (Arsitektur Berbasis Fitur/Halaman) yang dikombinasikan dengan pendekatan **Separation of Concerns (SoC)**.

Pola ini dipilih untuk memisahkan secara tegas antara lapisan presentasi (HTML), desain visual (CSS), dan logika fungsional pengendali data (JavaScript) untuk setiap fitur utama aplikasi. Pemisahan ini memastikan bahwa kode program dapat dikembangkan secara paralel, diuji secara independen, serta mempermudah proses pelacakan error jika terjadi kendala pada fitur tertentu.

Berikut adalah visualisasi pohon direktori dari organisasi berkas *frontend* BillMate:

BillMate Frontend

css/	# Stylesheet
index.css	# Halaman utama
createroom.css	# Halaman Create Room
room.css	# Halaman Room
history.css	# Halaman Riwayat
nota.css	# Halaman Nota
js/	# JavaScript
api.js	# Konfigurasi & pemanggilan API
index.js	# Logika halaman utama
createroom.js	# Logika Create Room
room.js	# Logika Room
history.js	# Logika Riwayat
nota.js	# Logika Nota
pages/	# HTML Pages
index.html	# Landing Page
createroom.html	# Create Room
room.html	# Room
history.html	# History
nota.html	# Nota
README.md	# Dokumentasi proyek

II.4.2. Implementasi Halaman & Komponen

a. Implementasi Halaman Utama

Aplikasi **BillMate** memiliki lima halaman utama yang terintegrasi secara sekuensial untuk mendukung alur pencatatan dan perhitungan tagihan, yaitu sebagai berikut.

- **Halaman Landing (index.html)**

Berfungsi sebagai pintu masuk utama (*welcome screen*) yang menyajikan ringkasan fitur utama aplikasi serta tombol untuk memulai pembuatan ruang baru dengan animasi transisi yang mulus.

- **Halaman Pembuatan Ruang (createroom.html)**

Menyediakan formulir masukan bagi **Host** untuk menginisialisasi sesi

baru. Host diwajibkan mengisi nama mereka, nama restoran, dan nomor rekening pembayaran sebelum masuk ke dasbor utama.

- **Halaman Dasbor Ruang Aktif (room.html)**

Merupakan ruang kerja utama di mana pengguna dapat mengelola anggota kelompok, menambah pesanan secara langsung (harga, jumlah, pemesan), serta memasukkan biaya penyesuaian seperti pajak, diskon, dan ongkos tambahan.

- **Halaman Nota Digital (nota.html)**

Merupakan halaman ringkasan akhir setelah proses kalkulasi selesai dilakukan. Halaman ini menampilkan detail porsi tagihan setiap anggota secara proporsional, serta dilengkapi tombol salin nomor rekening secara asinkron dan tombol berbagi langsung ke WhatsApp.

- **Halaman Riwayat (history.html)**

Menyajikan arsip transaksi kalkulasi yang disimpan secara lokal pada peramban. Halaman ini dilengkapi dengan fitur pencarian, filter rentang waktu, serta navigasi halaman (*pagination*).

b. Komponen Reusable

Untuk meminimalkan redundansi kode dan meningkatkan performa antarmuka, aplikasi memanfaatkan pembuatan elemen DOM secara dinamis dan modular. Komponen utama yang dirancang agar dapat digunakan kembali (*reusable*) adalah Item Order Card (Kartu Detail Pesanan Aktif).

Komponen ini diimplementasikan melalui fungsi helper `createItemCardElement` yang terdapat pada berkas `room.js`. Fungsi tersebut bertugas menghasilkan kartu daftar pesanan secara dinamis pada antarmuka pengguna sekaligus mengikat (*binding*) logika interaksi untuk tombol edit dan hapus pada setiap item.

Cuplikan kode komponen kunci:

```
1 cfunction createItemCardElement(foodName, foodPrice, foodQty, foodUser) {
2   const itemCard = document.createElement('div');
3
4   itemCard.className = "item-card flex justify-between items-center bg-surface-container-low p-sm rounded-2xl shadow-sm border border-surface-variant/30";
5
6   // Menyimpan metadata pesanan ke dalam data-attributes
7   itemCard.setAttribute('data-name', foodName);
8   itemCard.setAttribute('data-price', foodPrice);
9   itemCard.setAttribute('data-qty', foodQty);
10  itemCard.setAttribute('data-user', foodUser);
11
12  itemCard.innerHTML = `
13    <div class="flex items-center gap-sm">
14      <span class="material-symbols-outlined text-primary-container bg-primary-fixed/28 p-sm rounded-xl">restaurant</span>
15      <div>
16        <h4 class="font-bold text-on-surface text-sm item-name-text">${foodName}</h4>
17        <p class="text-xs text-secondary font-medium item-details-text">${foodUser} • Rp ${parseFloat(foodPrice).toLocaleString('id-ID')}</p>
18      </div>
19    </div>
20    <div class="flex items-center gap-md pr-sm">
21      <span class="font-extrabold text-primary-container bg-primary-fixed/38 px-md py-sm rounded-xl text-xs item-qty-text">x${foodQty}</span>
22      <div class="flex items-center gap-xs">
```

II.4.3. Kolaborasi

1. Konsep Arsitektur Kolaborasi

Sistem aplikasi **BillMate** dibangun menggunakan arsitektur *Decoupled Client-Server*, di mana bagian *Frontend* (antarmuka pengguna) dan *Backend* (mesin pemroses) berdiri secara terpisah namun saling berkolaborasi secara asinkron (*asynchronous*).

Dalam arsitektur ini, kolaborasi dibagi menjadi dua peran utama:

- **Sisi Klien (*Client-Side Frontend*):**

Berperan sebagai representasi visual dan interaksi langsung dengan pengguna. Frontend bertugas mengumpulkan masukan data transaksi (nama restoran, daftar pesanan, kuantitas, harga, dan penetapan pembeli), menangani transisi state UI (seperti animasi loading), dan menyajikan hasil nota akhir yang telah dihitung.

- **Sisi Server (*Server-Side Backend*):**

Berperan sebagai pusat mesin komputasi (*calculation engine*) berbasis kerangka kerja Laravel. Backend bertanggung jawab untuk menerima data mentah dari frontend, melakukan validasi keamanan data masukan, menjalankan kalkulasi matematis (proporsi pajak restoran, pembagian diskon flat/persentase, pembulatan desimal mata uang rupiah), menetapkan ID transaksi unik, dan menyimpan arsip transaksi jika diperlukan.

2. Protokol dan Media Kolaborasi

Untuk menjembatani komunikasi data antara frontend dan backend, kolaborasi diimplementasikan dengan memanfaatkan teknologi standar industri berikut:

- **Protokol HTTP & REST API:**

Komunikasi dijalankan melalui protokol komunikasi HTTP (*Hypertext Transfer Protocol*) berbasis REST (*Representational State Transfer*). Frontend mengirimkan data transaksi menggunakan metode **POST** ke *endpoint* API spesifik, yaitu `/api/split-bill/calculate`. Metode POST dipilih agar muatan data yang dikirimkan terbungkus di dalam *request body* demi keamanan dan kapasitas data yang lebih besar dibandingkan metode GET.

- **Format Data JSON (*JavaScript Object Notation*):**

Media pertukaran data yang digunakan adalah **JSON**. Format ini digunakan sebagai format standar karena strukturnya yang ringan, mudah dibaca manusia (*human-readable*), serta didukung penuh oleh JavaScript pada frontend dan bahasa pemrograman PHP pada Laravel di backend tanpa memerlukan proses penerjemahan data yang rumit.

- **Fetch API dengan Paradigma Asinkron (*Async/Await*):**

Di sisi frontend, permintaan jaringan dilakukan secara asinkron (*asynchronous HTTP request*) menggunakan **Fetch API** bawaan peramban modern. Sintaks `async/await` diterapkan untuk menyederhanakan penanganan *Promise*,

meminimalkan terjadinya *callback hell*, dan mengoptimalkan penanganan kesalahan (*error handling*) melalui blok kontrol try-catch.

- **API Gateway Pattern ([api.js](#)):**

Seluruh konfigurasi jaringan, penentuan *endpoint*, setelan header (Content-Type dan Accept), serta penanganan respons HTTP diisolasi secara terpusat di dalam kelas BillMateAPI. File antarmuka lain seperti *room.js* hanya perlu memanggil metode enkapsulasinya tanpa perlu menulis ulang konfigurasi koneksi server.

3. Justifikasi Pemilihan Arsitektur

Penerapan kolaborasi frontend-backend terpisah melalui API ini didasarkan pada analisis teknis dan arsitektur perangkat lunak berikut:

- **Pemisahan Tanggung Jawab (*Separation of Concerns*):**

Pemisahan ini membatasi kompleksitas kode. Tim pengembang frontend dapat fokus pada kualitas pengalaman pengguna (*User Experience*) dan responsivitas desain, sementara tim backend dapat fokus pada performa query basis data dan optimalisasi algoritma matematika tanpa adanya ketergantungan langsung (*coupling*) satu sama lain.

- **Keamanan dan Integritas Perhitungan:**

Penghitungan tagihan rawan dimanipulasi jika seluruhnya dieksekusi di sisi klien (browser pengguna), misalnya melalui penyuntikan nilai variabel secara langsung pada *browser console*. Dengan melemparkan data mentah ke server backend, **seluruh logika matematika dan validasi dijalankan di lingkungan server yang aman**, sehingga hasil perhitungan nota tidak dapat dicurangi.

- **Skalabilitas Multi-Platform:**

REST API bersifat agnostik terhadap platform klien. Backend Laravel yang sama dapat melayani berbagai macam klien di masa depan. Jika sistem ini dikembangkan menjadi aplikasi mobile Android/iOS, bot WhatsApp, atau aplikasi desktop, tim pengembang tidak perlu membangun kembali modul kalkulasi backend, melainkan cukup menghubungkannya ke API yang sudah ada.

- **Optimalisasi Kinerja Klien (*Client-Side Performance*):**

Pemrosesan matematika bertingkat dan pengelolaan basis data membutuhkan daya komputasi yang stabil. Dengan memindahkan beban pemrosesan (*heavy lifting*) ke server backend, beban kerja peramban di perangkat ponsel pintar pengguna yang berspesifikasi rendah tetap terjaga optimal, sehingga aplikasi tetap berjalan dengan lancar.

II.5. Backend Development

II.5.1. Struktur Project

a. Framework dan Versi

Proyek Bill Mate backend dikembangkan menggunakan **Laravel 13.8** dengan **PHP 8.3**. Laravel dipilih sebagai framework backend karena menyediakan ekosistem yang lengkap untuk pengembangan API, termasuk sistem routing, validasi, ORM (Eloquent), dan manajemen database migration.

b. Arsitektur Proyek

Proyek ini mengikuti pola arsitektur **MVC (Model-View-Controller)** bawaan Laravel dengan struktur *slim application* yang diperkenalkan sejak Laravel 11. Pada struktur slim ini, file `app/Http/Kernel.php` sudah tidak digunakan lagi — konfigurasi middleware dan routing dipusatkan pada file `bootstrap/app.php`.

Struktur Direktori Utama

```
Bill_Mate-backend/
├── app/
│   ├── Http/
│   │   └── Controllers/
│   │       └── API/
│   │           └── RoomController.php      ← Controller utama
│   ├── Models/
│   │   ├── Transaction.php               ← Model tabel transactions
│   │   ├── TransactionItem.php           ← Model tabel transaction_items
│   │   ├── TransactionMember.php         ← Model tabel transaction_members
│   │   └── User.php                      ← Model bawaan Laravel
│   └── Providers/
│       └── AppServiceProvider.php        ← Service provider utama
├── bootstrap/
│   └── app.php                          ← Konfigurasi aplikasi (routing,
│                                       middleware, exception)
├── config/
│   ├── app.php                          ← Konfigurasi umum aplikasi
│   └── auth.php                         ← Konfigurasi autentikasi
```


database.php	← Konfigurasi koneksi database
sanctum.php	← Konfigurasi Laravel Sanctum
database/ migrations/	← File migrasi skema database
public/ swagger/ openapi.json	← Spesifikasi OpenAPI 3.0.3
resources/ views/ swagger.blade.php	← Halaman Swagger UI
welcome.blade.php	← Halaman welcome default
routes/ api.php	← Definisi endpoint API
web.php	← Definisi route web
console.php	← Perintah Artisan kustom
composer.json	← Deklarasi dependensi PHP
.env	← Konfigurasi lingkungan
(environment)	

Penjelasan Peran Setiap Lapisan MVC

Lapisan	Direktori	Peran
Model	app/Models/	Merepresentasikan tabel database dan mendefinisikan atribut yang dapat diisi (<i>fillable</i>). Terdapat 3 model utama: Transaction, TransactionMember, dan TransactionItem.
View	resources/views/	Menyediakan tampilan web. Pada proyek ini hanya terdapat 2 view: halaman welcome dan halaman dokumentasi Swagger UI.
Controller	app/Http/Controllers/API/	Menangani logika bisnis dan mengelola alur request-response. Seluruh logika bisnis terkonsentrasi pada satu controller, yaitu RoomController.

Dependensi Utama (composer.json)

Package	Versi	Fungsi
laravel/framework	^13.8	Framework utama Laravel
laravel/sanctum	^4.0	Sistem autentikasi berbasis token API
laravel/tinker	^3.0	REPL interaktif untuk debugging dan eksplorasi

II.5.2. Implementasi API Endpoint

a. Controller: RoomController

Seluruh logika API dikelola oleh satu controller utama, yaitu

RoomController.php yang berada dalam namespace App\Http\Controllers\API.

Controller ini memiliki **2 method** yang masing-masing menangani satu endpoint:

b. Endpoint 1: calculateSplitBill()

Method ini menangani endpoint POST /api/split-bill/calculate dan merupakan **inti logika bisnis** aplikasi. Alur prosesnya adalah sebagai berikut:

1. **Validasi Input** — Menerima dan memvalidasi data JSON dari client menggunakan Validator::make() dengan aturan validasi terperinci (nama restoran, nomor meja, daftar anggota, daftar item pesanan, dan biaya tambahan).
2. **Kalkulasi Loop 1 (Biaya Pokok)** — Menghitung total biaya pokok (baseCost) per individu dengan menjumlahkan seluruh harga item pesanan milik masing-masing anggota.
3. **Kalkulasi Loop 2 (Pajak, Diskon, Grand Total)** — Untuk setiap anggota:
 - **Pajak** dihitung secara **proporsional** berdasarkan biaya pokok masing-masing anggota ($\text{baseCost} \times \text{taxPercent} / 100$).
 - **Diskon** dibagi secara **proporsional** berdasarkan rasio biaya pokok anggota terhadap total biaya pokok keseluruhan.
 - **Biaya tambahan** dibagi secara **rata (flat)** ke seluruh anggota.
 - **Grand total** = biaya pokok + pajak – diskon + biaya tambahan.
4. **Persistensi Database (DB::transaction)** — Data disimpan ke dalam 3 tabel secara atomik menggunakan DB::transaction() untuk menjamin konsistensi data (prinsip ACID):
 - Insert ke tabel transactions (data ringkasan transaksi)
 - Insert ke tabel transaction_members (rincian biaya per anggota)
 - Insert ke tabel transaction_items (detail item pesanan)
5. **Response JSON** — Mengembalikan respons JSON terstruktur berisi ringkasan transaksi, breakdown per anggota, dan informasi transfer pembayaran.

c. Endpoint 2: archiveRoom()

Method ini menangani endpoint POST `/api/split-bill/archive` dengan alur yang lebih sederhana:

- **Validasi Input** — Memvalidasi data yang dikirim (nama restoran, grand total, daftar anggota).
- **Generate ID Arsip** — Membuat ID unik arsip dengan format `#QR-2026-XXXX` menggunakan `random_bytes()`.
- **Sanitasi Data** — Menerapkan `strip_tags()` pada nama restoran untuk mencegah injeksi HTML/XSS.
- **Response JSON** — Mengembalikan *payload* arsip yang siap disimpan di sisi klien (`localStorage/frontend`).

[!NOTE] Endpoint archive **tidak menyimpan data ke database** — data arsip hanya dikembalikan sebagai respons JSON untuk disimpan oleh frontend secara lokal.

II.5.3. Autentikasi & Otorisasi (Authentication & Authorization)

a. Status Implementasi Autentikasi

Proyek Bill Mate telah menginstal package **Laravel Sanctum v4.0** sebagai sistem autentikasi. Sanctum menyediakan mekanisme autentikasi berbasis **token API (Personal Access Token)** yang ringan dan cocok untuk aplikasi SPA (Single Page Application) maupun mobile.

Infrastruktur pendukung autentikasi yang sudah tersedia dalam proyek meliputi:

Komponen	Status	Keterangan
Package laravel/sanctum	 Terinstal	Terdaftar di <code>composer.json</code>
Tabel <code>personal_access_tokens</code>	 Tersedia	Migrasi sudah dibuat untuk menyimpan token API
Tabel <code>users</code>	 Tersedia	Migrasi dan model User sudah tersedia

Konfigurasi config/sanctum.php	 Tersedia	Konfigurasi stateful domain dan middleware
Konfigurasi config/auth.php	 Tersedia	Guard dan provider autentikasi sudah dikonfigurasi

Saat ini, seluruh endpoint API beroperasi dalam mode **publik (tanpa autentikasi)**. Middleware auth:sanctum **belum diterapkan** pada route API manapun. Hal ini berarti siapa saja dapat mengakses endpoint POST /api/split-bill/calculate dan POST /api/split-bill/archive tanpa perlu menyertakan token autentikasi.

Definisi route saat ini pada file routes/api.php:

```
// Tanpa middleware auth – semua route bersifat publik
Route::post('/split-bill/calculate', [RoomController::class,
'calculateSplitBill']);
Route::post('/split-bill/archive', [RoomController::class,
'archiveRoom']);
```

[!IMPORTANT] Keputusan untuk tidak menerapkan autentikasi pada tahap ini diambil karena Bill Mate dirancang sebagai **kalkulator split-bill yang bersifat session-based**, di mana setiap sesi berdiri sendiri tanpa memerlukan akun pengguna. Namun, infrastruktur Sanctum telah disiapkan untuk mendukung fitur autentikasi di masa mendatang (misalnya: riwayat transaksi personal, multi-device sync, dsb).

II.5.4. Middleware & Keamanan (Middleware & Security)

a. Konfigurasi Middleware

Proyek Bill Mate menggunakan struktur **Laravel 11+ (slim application)** yang tidak lagi memiliki file app/Http/Kernel.php maupun direktori app/Http/Middleware/. Seluruh konfigurasi middleware dipusatkan pada file bootstrap/app.php:

```
return Application::configure(basePath: dirname(__DIR__))
->withRouting(
    web: __DIR__.'/../routes/web.php',
    api: __DIR__.'/../routes/api.php',
    commands: __DIR__.'/../routes/console.php',
    health: 'up',
)
->withMiddleware(function (Middleware $middleware): void {
    // Belum ada middleware kustom yang didaftarkan
```

```

    })
    ->withExceptions(function (Exceptions $exceptions): void {
        $exceptions->shouldRenderJsonWhen(
            fn (Request $request) => $request->is('api/*'),
        );
    }->create();

```

Meskipun tidak ada middleware kustom yang didaftarkan secara eksplisit, Laravel secara bawaan tetap menerapkan **middleware global** yang menjadi bagian dari framework, seperti penanganan CORS, enkripsi cookie, dan proteksi CSRF untuk route web.

b. Mekanisme Keamanan yang Diterapkan

Meskipun proyek ini tidak memiliki middleware kustom, beberapa praktik keamanan telah diimplementasikan secara langsung di dalam kode:

Aspek Keamanan	Implementasi	Lokasi
Validasi Input	Setiap endpoint menerapkan validasi ketat menggunakan <code>Validator::make()</code> dengan aturan tipe data, panjang maksimum, dan range nilai.	RoomController.php
Sanitasi Data	Penggunaan <code>strip_tags()</code> pada input <code>restaurantName</code> di endpoint <code>archive</code> untuk mencegah serangan XSS.	<code>archiveRoom()</code>
Database Transaction (ACID)	Penggunaan <code>DB::transaction()</code> untuk membungkus operasi insert multi-tabel, menjamin konsistensi data — jika salah satu operasi gagal, seluruh perubahan akan di-rollback.	<code>calculateSplitBill()</code>
Mass Assignment Protection	Setiap model Eloquent mendefinisikan properti <code>\$fillable</code> secara eksplisit, sehingga hanya kolom yang terdaftar yang dapat diisi melalui mass assignment.	Semua model
JSON Exception Handling	Konfigurasi <code>shouldRenderJsonWhen()</code> memastikan bahwa error pada route	<code>bootstrap/app.php</code>

	API (api/*) selalu dikembalikan dalam format JSON, bukan halaman HTML error.	
Cascading Delete	Foreign key dengan ON DELETE CASCADE memastikan integritas referensial — data orphan tidak akan tertinggal di tabel anak ketika data induk dihapus.	Migrasi database
Health Check Endpoint	Endpoint /up disediakan untuk monitoring status kesehatan server.	bootstrap/app.php

c. Dokumentasi API

Proyek ini menyediakan **dokumentasi API interaktif** menggunakan **Swagger UI** yang dapat diakses melalui route GET /api/docs. Spesifikasi API ditulis dalam format **OpenAPI 3.0.3** dan disimpan di file public/swagger/openapi.json. Halaman Swagger UI di-render menggunakan view Blade di resources/views/swagger.blade.php.

II.6. Tautan Repository

Link GitHub Repository:

<https://github.com/BillMate-in>

Link Dokumentasi API (Postman/Swagger):

<http://billmate-be.sao.dom.my.id/>

BAB III

KESIMPULAN

III.1. Rangkuman Proyek dan Pencapaian Tujuan

Pengembangan aplikasi **BillMate** berbasis web telah berhasil diselesaikan dengan mengintegrasikan antarmuka frontend yang responsif dan backend berbasis API Laravel 13 secara optimal. Proyek ini sukses menjawab permasalahan utama terkait kerumitan kalkulasi penagihan manual dalam aktivitas makan bersama kelompok. Melalui aplikasi ini, pengguna (khususnya *host*) dapat melakukan perhitungan pembagian tagihan secara otomatis, membagi biaya tambahan (pajak dan diskon secara proporsional, serta biaya layanan/ongkir dibagi rata) hanya dalam hitungan detik. Transparansi hasil perhitungan ini secara efektif meminimalisasi risiko kesalahan hitung (*human error*) dan menghindari potensi selisih angka yang merugikan.

III.2. Fitur dan Teknologi Menonjol

Komponen paling menonjol dalam pengembangan backend BillMate adalah **penerapan normalisasi database 3-Tier** yang dipadukan dengan fitur **Database Transaction (DB::transaction)** di Laravel. Mekanisme ini menjamin bahwa seluruh data ringkasan transaksi, detail anggota kelompok, dan rincian menu pesanan disimpan ke database MySQL secara atomik (prinsip ACID). Jika terjadi kegagalan sistem pada salah satu proses penyimpanan data anak, sistem secara otomatis akan melakukan *rollback* total untuk mencegah terjadinya data *orphan* (data yatim/tidak konsisten). Selain itu, penyediaan dokumentasi API interaktif menggunakan **Swagger UI** (GET /api/docs) berbasis spesifikasi **OpenAPI 3.0.3** mempermudah kolaborasi dan integrasi antara tim frontend dan backend.

III.3. Kendala yang Dihadapi dan Solusi

Selama proses pengerjaan proyek, beberapa kendala teknis berhasil diidentifikasi dan diselesaikan:

- **Kendala:** Kesulitan membagi potongan diskon nominal dan persentase pajak secara adil ketika pesanan setiap anggota kelompok memiliki nominal yang berbeda-beda.
 - **Solusi:** Merancang algoritma kalkulasi dua tahap (*two-pass calculation*) pada controller backend. Loop pertama mengunci biaya pokok individual, sedangkan loop kedua mendistribusikan pajak dan diskon secara proporsional sesuai rasio pengeluaran pokok masing-masing anggota.
- **Kendala:** Risiko hilangnya integritas data relasional akibat kegagalan koneksi database saat melakukan penyimpanan massal (*mass-insert*) ke banyak tabel anak secara simultan.
 - **Solusi:** Menggunakan penanganan transaksi database di Laravel (DB::transaction) untuk memastikan integritas data tetap terjaga seutuhnya.

III.4. Rencana Pengembangan Lebih Lanjut (*Future Development*)

Untuk meningkatkan performa dan fungsionalitas BillMate di masa mendatang, beberapa rencana pengembangan yang direkomendasikan adalah:

- **Integrasi OCR (Optical Character Recognition):** Menambahkan fitur pemindaian nota fisik menggunakan kamera untuk mendeteksi menu dan harga secara otomatis, sehingga pengguna tidak perlu memasukkan menu secara manual.
- **Aktivasi Autentikasi Laravel Sanctum:** Mengaktifkan modul autentikasi Sanctum yang sudah terpasang agar pengguna dapat mendaftar akun dan menyimpan riwayat transaksi secara permanen di server (*cloud sync*).
- **Integrasi Gerbang Pembayaran (*Payment Gateway*):** Menyediakan fitur pembayaran langsung/transfer instan melalui integrasi e-wallet atau bank transfer langsung dari aplikasi setelah hasil kalkulasi dibagikan.

III.5. *Insight* dan Pembelajaran Teknis

Pengerjaan proyek ini memberikan banyak pembelajaran berharga mengenai modernisasi ekosistem Laravel 11/13 yang mengusung konsep *slim structure* (tanpa

berkas Kernel.php tradisional). Tim pengembang backend memperoleh pemahaman yang lebih matang mengenai pentingnya pemisahan tanggung jawab (*separation of concerns*) melalui arsitektur API terpisah, teknik sanitasi input data untuk mencegah celah keamanan web seperti XSS/SQL Injection, serta pentingnya mendesain skema basis data yang terstruktur demi performa aplikasi yang responsif dan mudah dipelihara.